

Ejercicio Demostrativo

Manuel Lima

Tiempo estimado: 40 min

Universidad San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas.

Segundo Semestre 2026

Laboratorio – Introducción a la Programación y Computación – Sección P

1. Marco Formativo

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Al estructurar correctamente las clases, aplicar encapsulamiento y validar entradas antes de modificar el estado interno de los objetos, se demuestra responsabilidad en el diseño y la calidad del software.

1.1 Valores

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Aplica los principios de la programación orientada a objetos para modelar entidades del mundo real mediante clases, atributos, métodos y estructuras de datos nativas de C#.
Competencia Específica	Implementa un sistema de registro de estudiantes utilizando clases con encapsulamiento, List<T>, Dictionary<TKey, TValue>, arrays, tuplas y expresiones regulares, comprendiendo el rol de cada estructura en la solución.

2. Palabras Clave

Encapsulamiento: principio de POO que restringe el acceso directo a los atributos de una clase, exponiendo su comportamiento mediante métodos públicos (getters/setters).

List<T>: colección genérica dinámica de C# que permite agregar, eliminar y recorrer elementos del mismo tipo.

Dictionary<TKey, TValue>: colección de pares clave-valor que permite búsqueda en tiempo constante $O(1)$ por clave.

Tupla: estructura de C# que permite retornar múltiples valores desde un método en una sola instrucción.

Regex: clase de C# que permite validar cadenas de texto contra patrones definidos mediante expresiones regulares.

3. Resumen

Este ejercicio demostrativo muestra la construcción de un Sistema de Registro de Estudiantes en C#, compuesto por tres archivos: la clase Estudiante, la clase SistemaRegistro y el programa principal. El objetivo es que el estudiante comprenda cómo se combinan múltiples conceptos de POO y estructuras de datos nativas en un sistema funcional con menú interactivo.

La clase se orienta a que el estudiante observe cómo el encapsulamiento protege los datos internos, cómo `List<T>` y `Dictionary<TKey, TValue>` cumplen roles distintos en el mismo sistema, y cómo las expresiones regulares validan entradas antes de procesar información.

4. Contenido

Descripción del ejercicio

Se construye un sistema de registro de estudiantes universitarios. Cada estudiante tiene nombre, carnet y tres notas. El sistema permite agregar estudiantes con validación de carnet (9 dígitos), buscar por carnet y mostrar todos los registros. Al finalizar se espera un programa funcional con menú interactivo en consola.

Objetivo de la demostración

Demostrar el procedimiento para construir un sistema de registro en C# utilizando clases, encapsulamiento, `List<T>`, `Dictionary<TKey, TValue>`, tuplas y expresiones regulares, con el fin de que el estudiante comprenda cómo cada estructura resuelve un problema específico dentro de un mismo sistema.

Recursos necesarios

Los siguientes recursos son necesarios para ejecutar el ejercicio demostrativo:

- Visual Studio 2022 o Visual Studio Code con la extensión C# Dev Kit.
- .NET SDK 8.0 o superior. Consola del sistema para ejecutar el programa.
- Archivos de código fuente: Estudiante.cs, SistemaRegistro.cs, Program.cs.
- Computadora con sistema operativo Windows, macOS o Linux.
- Mínimo 4 GB RAM y proyecto de tipo Console Application en C#.

Desarrollo paso a paso

Nota: El procedimiento se divide en cuatro pasos que siguen el orden lógico de construcción del sistema. Cada archivo debe crearse en el mismo proyecto de C#. Se recomienda mostrar el código en pantalla mientras se explica cada concepto.

Cada paso incluye qué se hace, por qué se hace y qué resultado parcial se espera. Los comentarios en el código son parte de la explicación y deben leerse en voz alta durante la demostración.

Paso 1

Crear el proyecto Console Application en C# y agregar el archivo Estudiante.cs dentro del namespace SistemaEstudiantes. Declarar los atributos privados (nombre, carnet, notas, promedio) y el constructor que recibe los cinco parámetros. El estudiante debe observar cómo el array `int[]` notas se inicializa con tamaño fijo de 3 y se puebla en el constructor, y cómo el promedio se calcula automáticamente mediante un método privado con un ciclo `for`.

Paso 2

Agregar los getters, setters y el método `ObtenerEstadoAcademico()` a la clase Estudiante. Demostrar que `GetNotas()` retorna una copia del array con `Array.Copy()` para proteger los datos internos, y que `SetNota()` recalcula el promedio automáticamente tras la modificación. Enfatizar el método `ObtenerEstadoAcademico()` como ejemplo de tupla: retorna (`double` promedio, `string` estado) en una sola instrucción.

Paso 3

Crear el archivo SistemaRegistro.cs. Declarar `List<Estudiante>` y `Dictionary<string, Estudiante>` como atributos privados. Implementar `ValidarCarnet()` usando `Regex.IsMatch()` con el patrón `@'^\d{9}$'`, y `AgregarEstudiante()` que valida el formato, verifica duplicados con `ContainsKey()`, crea el objeto y lo agrega a ambas estructuras simultáneamente. Mostrar `BuscarPorCarnet()` con `TryGetValue()` como alternativa segura a indexar directamente el diccionario.

Paso 4

Crear Program.cs con el menú interactivo. Instanciar SistemaRegistro y ejecutar el ciclo `while` con `switch` para las tres opciones. Agregar al menos dos estudiantes desde consola, buscar uno por carnet y mostrar todos. Verificar que el sistema rechaza carnets con formato incorrecto y carnets duplicados. La confirmación de éxito es ver la información de cada estudiante correctamente impresa con el estado APROBADO o REPROBADO según el promedio.

Observaciones importantes

El Dictionary y la List mantienen referencia al mismo objeto Estudiante: modificar el objeto desde cualquiera de las dos estructuras afecta ambas. Si se usa `int.Parse()` sin manejo de excepciones el programa termina abruptamente al ingresar texto no numérico; en producción se debe usar

`int.TryParse()`. El patrón Regex `@'^\d{9}$'` es estricto: no acepta espacios ni guiones. Demostrar explícitamente el caso de carnet duplicado para que el estudiante vea el mensaje de error.

Resultado esperado

Al ejecutar el programa se muestra un menú interactivo en consola. Al agregar un estudiante con carnet de 9 dígitos y notas válidas, el sistema imprime "Estudiante agregado exitosamente." Al buscar por carnet, se despliega la información completa con nombre, carnet, notas, promedio y estado académico (APROBADO/REPROBADO). Al intentar agregar un carnet duplicado o con formato incorrecto, el sistema muestra el mensaje de error correspondiente sin terminar la ejecución.

Aplicación práctica

Este ejercicio es la base directa para las prácticas del laboratorio donde se implementan sistemas CRUD con persistencia. El patrón de usar List para iteración y Dictionary para búsqueda rápida es estándar en aplicaciones reales: lo usan frameworks como ASP.NET al gestionar sesiones o cachés. El encapsulamiento aplicado aquí es el mismo principio que protege los modelos de datos en arquitecturas MVC y microservicios.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- La mayor dificultad observada fue entender por qué se usan List y Dictionary simultáneamente para el mismo dato. Conviene explicarlo con la analogía: la List es el "archivo físico" (guarda todo en orden) y el Dictionary es el "índice" (permite encontrar rápido por carnet).
- Para futuras sesiones se recomienda agregar manejo de excepciones con try-catch en la lectura de notas, y mostrar cómo `int.TryParse()` es la alternativa robusta a `int.Parse()`.
- El concepto de mayor dificultad para el estudiante fue la tupla en `ObtenerEstadoAcademico()`: no estaban familiarizados con retornar múltiples valores en C#. Se recomienda mostrar primero la versión sin tupla (dos métodos separados) antes de introducir la sintaxis de tupla.

6. Referencias

1. Microsoft. (2024). Collections and Data Structures. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/collections/>
2. Microsoft. (2024). Encapsulation in C#. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/>